

003: Turn-PPO 与 StepPPO 对比分析 (Append-only Chat)

用户与 Codex

2026-06-08

记录约定

本文件是新的 Chat 归档文件，记录本轮关于 Turn-PPO 论文、仓库实现与当前 StepPPO 的对比分析。后续如继续讨论该主题，应在本文件末尾 append，不重写已有内容。公式使用 LaTeX 形式记录。

Turn 1 | User

有一篇Turn-PPO: Turn-Level Advantage Estimation with PPO for Improved Multi-Turn RL
→ in Agentic LLMs, 和我的想法很接近。/prj/corp/crd/morpheus/lasvegas/china-scratch/zi
→ yuhuan/Projects/verl-agent/EXPS下载这个论文，并查看它的git仓库。
→ 对比它和我的Step-PPO的不同之处。上边的那些分析，开一个新的/prj/corp/crd/morpheus/la
→ s Vegas/china-scratch/ziyuhuan/Projects/verl-agent/EXPS/Chat写进去

Turn 1 | Codex

本地材料与仓库状态

已下载和检查的材料如下：

- 论文 PDF: EXPS/papers/turn_ppo_2512.17008.pdf
- 论文 HTML: EXPS/papers/turn_ppo_2512.17008.html
- 论文 HTML 文本抽取: EXPS/papers/turn_ppo_2512.17008_html.txt
- 原始 RAGEN 仓库浅克隆: EXPS/repos/RAGEN, remote 为 <https://github.com/ml1-lab-nu/RAGEN>. git, 当前 commit 为 20daedc47558e000f7de912b060646bf2e8026bd。
- 搜索到的 Turn-PPO fork: EXPS/repos/RAGEN_turnPPO, remote 为 https://github.com/sjwjames/RAGEN_turnPPO. git, 当前 commit 同样为 20daedc47558e000f7de912b060646bf2e8026bd。

需要强调：论文正文没有给出一个明确的 GitHub URL，只写到训练代码基于 RAGEN 并实现了 Turn-PPO。当前搜索到的 sjwjames/RAGEN_turnPPO 与 ml1-lab-nu/RAGEN 的 main 分支在该 commit 下内容一致，diff -qr 未发现工作树差异。因此下面把“Turn-PPO 论文定义”和“公开 RAGEN 代码中可见的实现线索”分开讨论，避免把当前公开仓库误认为完整、最终的论文代码 release。

Turn-PPO 的核心定义

Turn-PPO 的核心不是“每个 token 都有一个独立 MDP step”，而是把一个 agent turn 建模成 MDP 的一个 step。对第 i 条 trajectory 的第 n 个 turn：

- state $s_{i,n}$ 是该 turn 生成前的上下文，也就是用户问题、历史交互和环境返回后的边界状态。
- action $a_{i,n}$ 是该 turn 的完整 LLM response，包括可能的 reasoning 和 action/tool call token：

$$a_{i,n} = (a_{i,n,1}, \dots, a_{i,n,H_{i,n}}).$$

- value $V_\phi(s_{i,n})$ 是 turn-level state value。论文描述中，value 使用该 turn 的 query/environment-output 最后 token 对应的 critic output。
- advantage 在 turn 维度用 GAE 计算，而不是在 token 维度计算：

$$\delta_{i,n} = r_{i,n} + \gamma V_\phi(s_{i,n+1}) - V_\phi(s_{i,n}),$$

$$A_{i,n}^{\text{turn}} = \sum_{\ell=0}^{N_i-n} (\gamma\lambda)^\ell \delta_{i,n+\ell}.$$

- return 为

$$\hat{R}_{i,n} = A_{i,n}^{\text{turn}} + V_\phi(s_{i,n}).$$

论文的 turn-level PPO ratio 是完整 response 的概率 ratio，而不是逐 token 独立 clipping：

$$\rho_{i,n}(\theta) = \frac{\pi_\theta(a_{i,n} | s_{i,n})}{\pi_{\theta_{\text{old}}}(a_{i,n} | s_{i,n})} = \prod_{h=1}^{H_{i,n}} \frac{\pi_\theta(a_{i,n,h} | s_{i,n}, a_{i,n,<h})}{\pi_{\theta_{\text{old}}}(a_{i,n,h} | s_{i,n}, a_{i,n,<h})}.$$

其 actor objective 可写为：

$$\max_{\theta} \mathbb{E} \left[\frac{1}{G} \sum_{i=1}^G \frac{1}{N_i} \sum_{n=1}^{N_i} \min(\rho_{i,n}(\theta) A_{i,n}^{\text{turn}}, \text{clip}(\rho_{i,n}(\theta), 1 - \epsilon, 1 + \epsilon) A_{i,n}^{\text{turn}}) \right].$$

critic 的 value loss 是 per-turn 的 PPO-style value objective：

$$\min_{\phi} \mathbb{E} \left[\frac{1}{G} \sum_{i=1}^G \frac{1}{N_i} \sum_{n=1}^{N_i} \frac{1}{2} (V_\phi(s_{i,n}) - \hat{R}_{i,n})^2 \right].$$

这个定义和你最初提出的“每个 action/step 只产生一个 value，GAE/TD-error/return 在 step 维度计算”的方向非常接近。更准确地说，Turn-PPO 是 state-value 版本：value 是 $V(s_{i,n})$ ，不是 action-conditioned 的 $Q(s_{i,n}, a_{i,n})$ 。因此取“生成前边界 value”比“第一个生成 token 的 value”更严格，也更符合论文定义。

公开 RAGEN 代码里的实现线索

公开 RAGEN 代码中最接近 Turn-PPO 的函数是：

`EXPS/repos/RAGEN/ragen/trainer/core_algos.py::compute_bi_level_gae_advantage_return`

该函数做了两层 GAE：

1. high-level: 在每个 turn 的 reward/EOS 位置上计算 per-turn GAE。代码注释写明 high level 是 per-turn wise。

2. low-level: 再把 high-level return 写回对应 EOS 位置, 沿当前 response 的 token 位置做一遍 token-level GAE。最后对 token-level advantages 做 `masked_whiten`。

这说明公开代码仍保留了 token-level loss path 的兼容处理; 它并不是一个非常干净的“只在 turn-level ratio 上 clipping”的论文公式实现。另外, `EXPS/repos/RAGEN/train.py` 中还有如下约束:

$$\text{bi_level_gae}=\text{True} \Rightarrow \text{adv_estimator}=\text{gae},$$

并且当前 main 分支断言 `use_turn_scores` 在 `bi_level_gae` 下尚未正确支持。这进一步说明公开仓库不一定是论文最终 Turn-PPO 代码的完整状态。

当前 StepPPO-v2 的实际定义

当前 StepPPO-v2 代码主要位于:

- `verl/trainer/ppo/step_ppo_algos.py`
- `verl/workers/actor/step_ppo_actor.py`
- `exps/run_webshop_step_ppo_v2_4gpu_paper_align_simple.sh`

其 step-wise GAE 是:

$$\begin{aligned}\delta_{i,t}^{\text{step}} &= r_{i,t}^{\text{step}} + \gamma V_{\psi}(s_{i,t+1}) - V_{\psi}(s_{i,t}), \\ A_{i,t}^{\text{step}} &= \delta_{i,t}^{\text{step}} + \gamma \lambda A_{i,t+1}^{\text{step}}.\end{aligned}$$

其中 trajectory 用 `traj_uid` 分组, step 顺序用 `step_idx` 排序。value 取 `pre_action_last_context_token`, 也就是生成 action 前的边界 token; 这点和 Turn-PPO 的 state-value 边界选择基本一致。

但 StepPPO-v2 不是纯 PPO advantage。它把 GiGPO-style episode advantage 与 step GAE advantage 混合:

$$A_{i,t}^{\text{final}} = A_{i,t}^{\text{epi}} + w \hat{A}_{i,t}^{\text{step}}, \quad w = 0.5.$$

当前脚本中对应配置为:

```
algorithm.step_ppo.step_advantage_w=0.5
algorithm.step_ppo.episode_mode=mean_norm
algorithm.step_ppo.final_advantage_mode=direct
algorithm.step_ppo.normalize_episode_advantage=False
algorithm.step_ppo.normalize_step_advantage=True
algorithm.step_ppo.normalize_final_advantage=False
actor_rollout_ref.actor.value_head.detach_value_backbone=False
actor_rollout_ref.actor.value_head.value_loss_coef=0.03
actor_rollout_ref.actor.ppo_micro_batch_size_per_gpu=8
```

这里的 critic 不是另一个单独的 model, 而是 actor backbone 加一个 value MLP head。value loss 和 policy loss 在同一个 actor update path 中合并:

$$\mathcal{L} = \mathcal{L}_{\text{policy}} + c_v \mathcal{L}_{\text{value}}, \quad c_v = 0.03.$$

另外, 当前 StepPPO-v2 的 actor loss path 仍是 verL 的 vanilla token-level PPO loss。即便每个 step 的 scalar advantage 被 broadcast 到该 response 的所有 token, ratio 和 clipping 仍在 token 维度做:

$$\rho_{i,t,h}(\theta) = \exp(\log \pi_{\theta}(a_{i,t,h}) - \log \pi_{\theta_{\text{old}}}(a_{i,t,h})).$$

这与 Turn-PPO 论文的 response-level/turn-level ratio 不同。

Turn-PPO 与当前 StepPPO-v2 的主要差异

维度	Turn-PPO	当前 StepPPO-v2
MDP step	一个 turn 是一个 MDP step, action 是完整 LLM response。	一个 environment/action step 是一个 step, 形式上与 turn 接近。
value 位置	生成前 state 边界; 论文说取 query/environment-output 最后 token 的 critic output。	<code>pre_action_last_context_token</code> , 基本对齐生成前边界。
advantage 来源	纯 critic/GAE: A^{turn} 。	hybrid: $A^{\text{epi}} + 0.5\hat{A}^{\text{step}}$ 。
episode term	没有 GRPO/GiGPO episode-relative advantage 混合。	保留 GiGPO-style <code>mean_norm</code> episode term。
normalization	PPO 中通常对 GAE advantage 做 whitening; 论文重点不依赖 group-relative reward normalization。	step advantage batch-level whitening; episode advantage 用 uid group mean centering; final 不 whitening。
policy ratio/clipping	response/turn-level ratio: 完整 response 概率比值或其 geometric average 变体。	默认 <code>policy_loss.loss_mode=vanilla</code> , token-level ratio/clipping。
critic 结构	论文说 critic 初始化自 pretrained LLM, 并 attach separate value head; actor/critic 学习率分开, critic LR 约为 actor 的 5–10 倍。	actor 自身加 value head, 共享 backbone; 当前无独立 critic optimizer/LR。
rollout group	WebShop PPO 超参中 $G = 1$, 强调 PPO 更需要 prompt diversity 而非每题多 rollouts。	继承 WebShop StepPPO 脚本 <code>env.rollout.n=8</code> , 仍是 grouped rollout 结构。
奖励口径	turn reward 应进入 turn-level GAE。	step reward 已包含 invalid action penalty; 但 episode term 优先读 <code>non_tensor_batch["episode_rewards"]</code> , 存在不含 invalid penalty 的风险。

最关键的不同可以压缩成三点:

1. Turn-PPO 是“纯 turn-level PPO/GAE”; 当前 StepPPO-v2 是“GiGPO episode advantage + step GAE correction”的 hybrid。
2. Turn-PPO 的 clipping 单位是整个 response/turn; 当前 StepPPO-v2 的 clipping 单位仍是 token。
3. Turn-PPO 使用更传统的 actor-critic 分离训练假设; 当前 StepPPO-v2 使用 shared-backbone value head, 把 value loss 直接加进 actor update。

从 Turn-PPO 视角重新解释当前 StepPPO-v2 失败

当前 StepPPO-v2 最近一次完整训练日志为:

`logs/webshop_step_ppo_qwen25_15b_4gpu_paper_align_simple_20260605_031621.log`

该 run 完成到 step 250, 没有工程崩溃, 但效果明显弱于 GiGPO:

- final validation: `val/success_rate=0.546875, val/webshop_task_score=0.6619, val/text/test_score=3.485`。
- best validation 大约在 step 170 附近, 之后回落, 说明存在 over-training 或 advantage/value signal 失真。

- 训练末期 `episode/valid_action_ratio` 很低，最终约 0.345，last-20 平均约 0.274；而 GiGPO 三个 seed 的 last-20 `valid_action_ratio` 约 0.996--0.998。
- `response_length/clip_ratio` 在 StepPPO-v2 后期明显升高，last-20 平均约 0.088；GiGPO 大约 0.001 量级。

这不是一个简单的 value loss 大小问题，而是 policy 行为已经偏离 WebShop action format：模型倾向输出长文本、中文 reasoning 或 invalid action，导致 valid action ratio 学不起来。Turn-PPO 论文本身指出，多 turn GRPO 的一个问题是 uniform trajectory advantage 不能区分不同 turn 的贡献；但 StepPPO-v2 的失败更具体：引入 step GAE 后，仍没有让 action-format 学习稳定下来。

目前最可疑的实现问题是 episode advantage 的 reward 口径。`verl/trainer/ppo/step_ppo_algos.py` 中 StepPPO 优先从：

```
data.non_tensor_batch["episode_rewards"]
```

读取 episode score；而 invalid action penalty 是之后写到 `token_level_scores` 和 `step_rewards` 里的。也就是说：

- A^{step} 能看到 invalid action penalty；
- A^{epi} 可能仍然使用未惩罚的 raw episode reward；
- v2 又采用

$$A^{\text{final}} = A^{\text{epi}} + 0.5\hat{A}^{\text{step}},$$

因此 raw episode term 可能主导优化方向，step correction 不足以压制 invalid action。

这个问题与 Turn-PPO 的定义不一致。Turn-PPO 的 GAE reward 应该是用于训练的 turn reward；如果 invalid action penalty 是环境反馈的一部分，它必须进入 $r_{i,n}$ ，否则 critic 和 advantage 会学习错误的 transition value。

第二个可疑点是 final advantage scale。当前 v2 的 episode term 是 `mean_norm`，其尺度可能仍接近 task reward 差值；step term 被 batch whitening 后是 unit-scale，再乘 0.5。如果 episode term 的数值尺度更大，实际优化会更像 GiGPO episode-only 的变体，而不是一个有效的 step-wise PPO。Turn-PPO 则不会混入一个 group-relative episode term，所以这个 scale conflict 不存在。

第三个差异是 clipping 粒度。Turn-PPO 强调完整 response 是 action，因此 PPO ratio/clipping 应该在 turn/action 维度计算。当前 StepPPO-v2 虽然 advantage 是 step scalar，但 token-level clipping 会允许同一个 action 内不同 token 受到不同 clipping 状态，严格说仍是 token-MDP loss。这可能削弱“一个 action 一个 advantage”的一致性。

第四个差异是 critic 优化。Turn-PPO 论文强调 actor/critic LR 非常敏感，并使用 actor LR 10^{-6} 、critic LR 10^{-5} 。当前 StepPPO-v2 的 value head 和 actor 共用 optimizer path；即使 `value_loss_coef=0.03`，critic 仍没有独立、更快的学习率。日志中后期 `actor/value_loss` 仍经常处于较高水平，说明 value baseline 可能没有足够快地追上 step return。

对下一版 StepPPO 的建议

如果目标是“更接近 Turn-PPO”，建议单独实现一个 `turn_ppo` 或 `step_ppo_pure` 版本，不再继续在当前 hybrid v2 上叠补丁：

1. 去掉 A^{epi} 混合，先做纯 step/turn GAE：

$$A_{i,t}^{\text{final}} = \text{Whiten}(A_{i,t}^{\text{step}}).$$

2. 保证 $r_{i,t}^{\text{step}}$ 包含 invalid action penalty、KL shaping 和所有用于 policy learning 的 reward shaping；不要让 episode term 或 return 读取未惩罚 raw reward。
3. 将 policy loss 从 token-level clipping 切换到 response/action-level clipping。当前代码里已有 `compute_policy_loss_gs` 它用 response 的 geometric average ratio，可能是实现 Turn-PPO action-level clipping 的最小改动起点；但最好新建 StepPPO/TurnPPO 专用 loss，显式以每个 step response 为 action。
4. 对 PPO WebShop 先把 group size 对齐 Turn-PPO 论文思路，尝试 `env.rollout.n=1`，用更高 prompt diversity 替代每题多 rollouts。GiGPO baseline 继续使用多 rollouts。
5. critic 训练需要更强的独立性：最低限度要给 value head 单独参数组和更大学习率；更严格地则维护 separate critic model。若坚持 shared backbone，建议先试 `detach_value_backbone=True` 或 value-head-only warmup，避免 value loss 干扰 action format。
6. 增加诊断 metrics: A^{epi} 、 A^{step} 、 A^{final} 的均值/方差/分位数，valid vs invalid action 的 advantage 均值，value explained variance，response-level ratio/clipfrac。

如果目标是“保留 GiGPO-style hybrid StepPPO”，则至少应先修正两个问题：

1. A^{epi} 必须从 penalized episode score 或 `token_level_rewards.sum(-1)` 计算，不能使用未含 invalid penalty 的 raw `episode_rewards`。
2. 重新标定 A^{epi} 与 A^{step} 的尺度；否则 $w = 0.5$ 只是名义权重，不代表实际优化权重。

结论

Turn-PPO 与你的原始 StepPPO 想法确实很接近：二者都认为 agentic multi-turn 任务应在 action/turn/step 维度做 credit assignment，而不是把所有 token 当作等价 MDP step。当前 StepPPO-v2 已经在 value 边界和 step-level GAE 上接近 Turn-PPO，但仍有三个本质偏差：

1. 它仍混入 GiGPO/GRPO episode-relative advantage；
2. 它的 policy ratio/clipping 仍是 token-level；
3. 它的 critic 是 shared actor value head，且没有独立 critic LR。

因此，当前 StepPPO-v2 不能被视为 Turn-PPO 的复现。它更像一个“GiGPO episode advantage + shared-head step GAE correction”的 hybrid 方法。若要做和 Turn-PPO 更公平的比较，应该新增一个 pure Step/Turn PPO 版本，把 advantage、reward、ratio/clipping、critic optimizer 都严格对齐到 turn-level PPO 定义。

Turn 2 | User

看一下实现的细节，value 的 loss 计算的时候是否 detach 了，value head 计算的 A 是否 detach 了

Turn 2 | Codex

结论：当前实现中，advantage 和 return 是 detached 的；value loss 的 target/old value 也是 detached 的；但 actor update 阶段重新计算得到的 current value prediction 默认不 detach。当前 v2 配置为 `detach_value_backbone=False`，所以 value loss 会同时反传到 value head 和 actor backbone。

old state values 是 detached 的

old-log-probability 路径会同时计算 `old_log_probs` 和 `state_values`。在 `verl/workers/actor/step_ppo_actor.py` 中, `compute_log_prob` 的 micro-batch forward 被包在:

```
with torch.no_grad():
```

因此, 这一步生成的 `state_values` 是 old value, 只作为后续 GAE 的固定输入, 不带梯度。

数学上, 这一步得到的是:

$$V_{\text{old}}(s_t) = V_{\psi_{\text{old}}}(s_t),$$

并且在实现中等价于:

$$\text{stopgrad}(V_{\text{old}}(s_t)).$$

Step advantage / return 计算是 detached 的

StepPPO 的 GAE 位于 `verl/trainer/ppo/step_ppo_algos.py::compute_step_gae_advantage_return`。该函数内部显式使用 `torch.no_grad()`:

```
with torch.no_grad():
```

```
    delta = rewards[idx] + gamma * next_value * nonterminal - old_values[idx]
    lastgaelam = delta + gamma * lam * nonterminal * lastgaelam
    advantages[idx] = lastgaelam
    returns[idx] = advantages[idx] + old_values[idx]
```

所以 step-level TD-error、GAE advantage 和 return 都不会把梯度传回 value head:

$$\delta_t = r_t + \gamma V_{\text{old}}(s_{t+1}) - V_{\text{old}}(s_t),$$

$$A_t^{\text{step}} = \delta_t + \gamma \lambda A_{t+1}^{\text{step}},$$

$$\hat{R}_t = A_t^{\text{step}} + V_{\text{old}}(s_t).$$

在实现语义上可以写成:

$$A_t^{\text{step}} = \text{stopgrad}(A_t^{\text{step}}), \quad \hat{R}_t = \text{stopgrad}(\hat{R}_t).$$

Episode advantage 也是 detached 的

GRPO-style 和 GiGPO-style episode advantage 的计算函数也都在 `torch.no_grad()` 内部执行。因此:

$$A_t^{\text{epi}} = \text{stopgrad}(A_t^{\text{epi}}).$$

最终 StepPPO-v2 的 mixed advantage 为:

$$A_t^{\text{final}} = A_t^{\text{epi}} + w \hat{A}_t^{\text{step}}, \quad w = 0.5,$$

其中 A_t^{epi} 和 $\hat{A}_t^{\text{step}}$ 都是 detached training signal。代码中 `advantages = final_step_advantages.unsqueeze(-1) * response_mask` 只是把 step scalar broadcast 到 response tokens, 不会重新引入 value-head 梯度。

value loss 的 current value 不 detach

actor update 阶段会重新 forward 一次 actor/value-head, 得到 current values。value loss 使用:

```
current_values = current_values.float()
old_values = micro_data["state_values"].float()
step_returns = micro_data["step_returns"].float()

values_clipped = old_values + torch.clamp(
    current_values - old_values,
    -cliprange_value,
    cliprange_value,
)
value_loss_unclipped = (current_values - step_returns).pow(2)
value_loss_clipped = (values_clipped - step_returns).pow(2)
value_loss = 0.5 * torch.max(value_loss_unclipped, value_loss_clipped).mean()
policy_loss = policy_loss + value_loss_coef * value_loss
```

这里的梯度关系是:

- `old_values` 来自 old-log-prob 路径, 已经 detached;
- `step_returns` 来自 GAE 路径, 已经 detached;
- `current_values` 来自当前 actor update forward, 不 detach;
- 因此 `value_loss` 会对 `current_values` 反传。

对应公式为:

$$\mathcal{L}_V = \frac{1}{2} \max \left(\left(V_\psi(s_t) - \text{stopgrad}(\hat{R}_t) \right)^2, \left(\text{clip}(V_\psi(s_t), V_{\text{old}}(s_t) - \epsilon_v, V_{\text{old}}(s_t) + \epsilon_v) - \text{stopgrad}(\hat{R}_t) \right)^2 \right).$$

因此:

$$\nabla_\psi \mathcal{L}_V \neq 0.$$

是否反传到 backbone 由 detach_value_backbone 控制

value head 的 forward 在 `verl/workers/actor/value_head.py` 中实现:

```
hidden_states = output.hidden_states[-1]
if bool(getattr(self, "_step_value_head_detach_backbone", False)):
    hidden_states = hidden_states.detach()
step_values = self.value_head(hidden_states)
```

因此:

- 若 `detach_value_backbone=True`, value loss 只更新 value head, 不通过 value path 更新 actor backbone;
- 若 `detach_value_backbone=False`, value loss 同时更新 value head 和 actor backbone。

当前 v2 脚本 `exps/run_webshop_step_ppo_v2_4gpu_paper_align_simple.sh` 中设置:

```
actor_rollout_ref.actor.value_head.detach_value_backbone=False
```

所以当前 v2 的实际梯度为：

$$\nabla_{\theta_{\text{backbone}}} \mathcal{L}_V \neq 0, \quad \nabla_{\psi_{\text{value_head}}} \mathcal{L}_V \neq 0.$$

如果改为 `detach_value_backbone=True`，则变成：

$$\nabla_{\theta_{\text{backbone}}} \mathcal{L}_V = 0, \quad \nabla_{\psi_{\text{value_head}}} \mathcal{L}_V \neq 0.$$

policy loss 仍然会正常更新 actor backbone。

一句话总结

当前 StepPPO 实现是 PPO-style 的 stop-gradient 结构：A 和 $\hat{R}$ 都是 old policy/value 下的 detached 训练信号；value loss 的 $V_{\psi}(s_t)$ 不 detach。由于当前 v2 使用 `detach_value_backbone=False`，value loss 会反传到 shared actor backbone，这可能干扰 policy/action-format 学习，也是后续 ablation 中应重点检查的因素。

Turn 3 | User

2. 保证 r

step

i,t

包含 invalid action penalty、KL shaping 和所有用于 policy learning 的 reward shaping；

不要让 episode term 或 return 读取未惩罚 raw reward。这里的 reward shaping 具体是？

→ 写到 003 中

Turn 3 | Codex

这里的 reward shaping 指的是：在环境原始奖励 $r_{i,t}^{\text{env}}$ 之外，为了让 policy learning 感知格式、约束、偏好或稳定性而加入到 reward 空间里的所有修正项。它们不是独立的 logging metric，而是会进入 advantage/return 计算的训练奖励。对 StepPPO 来说，严格要求是：critic/GAE 使用的 $r_{i,t}^{\text{step}}$ 、episode term 使用的 score，以及 policy loss 间接受到的 return/advantage，必须基于同一套 shaped reward 口径。

一般形式

可以把 step reward 写成：

$$r_{i,t}^{\text{step}} = r_{i,t}^{\text{env}} + r_{i,t}^{\text{task_shape}} - \alpha \mathbf{1}\{a_{i,t} \text{ invalid}\} - \beta \sum_{h \in a_{i,t}} D_{\text{KL},i,t,h} + r_{i,t}^{\text{other_shape}}.$$

其中：

- $r_{i,t}^{\text{env}}$ ：环境返回的原始 step reward，例如 WebShop 环境在某一步返回的分数增量或最终得分信号。
- $r_{i,t}^{\text{task_shape}}$ ：任务侧 dense shaping，例如 step bonus、process reward、progress reward、长度归一化后的 reward 等。如果当前任务没有这些项，则为 0。
- $-\alpha \mathbf{1}\{a_{i,t} \text{ invalid}\}$ ：invalid action penalty，用于惩罚格式错误、非法 tool call 或环境无法执行的 action。
- $-\beta \sum_h D_{\text{KL},i,t,h}$ ：reward-space KL penalty。只有当 KL 被配置为 reward shaping，即 `algorithm.use_kl_in_reward=True` 时，它才属于 r^{step} 。

- $r_{i,t}^{\text{other_shape}}$: 其他真正进入 reward 的修正项, 例如 safety penalty、重复动作 penalty、action length penalty、tool-call cost 等。

这里的原则是: 只要某个项会改变 advantage/return, 就必须被视为 reward shaping; 只用于日志或只作为额外 loss 的项, 不属于 r^{step} 。

当前代码里的实际 reward 口径

当前 rollout 中, 环境每一步返回 rewards, 并写入:

```
batch.non_tensor_batch["rewards"]
```

同时累计得到 raw episode reward:

```
episode_rewards += rewards
```

reward manager 后续会把 episode_rewards 写到 token_level_scores 的最后一个 response token 上。也就是说, raw episode score 的来源是环境累计分数。

随后训练 loop 会先应用 invalid action penalty。当前实现会同时修改 token-level score 和 step reward:

```
reward_tensor[i, valid_response_length - 1] -= invalid_action_penalty_coef *
↪ action_invalids
step_rewards[i] -= invalid_action_penalty_coef * action_invalids
```

因此, 在当前配置下, invalid action penalty 是真正的 reward shaping。它应该进入:

$$r_{i,t}^{\text{step}} = r_{i,t}^{\text{env}} - 0.1 \cdot \mathbf{1}\{a_{i,t} \text{ invalid}\},$$

因为当前脚本里 invalid_action_penalty_coef=0.1。

KL shaping 和 actor KL loss 的区别

当前代码有两种 KL 使用方式, 必须区分:

1. reward-space KL penalty: 当 algorithm.use_kl_in_reward=True 时, 代码执行

```
token_level_rewards = token_level_scores - beta * kld
```

这属于 reward shaping, 因为它改变了用于 advantage 的 reward。

2. actor KL loss: 当 actor_rollout_ref.actor.use_kl_loss=True 时, KL 作为 policy loss 的额外正则项加入 actor update。这不是 reward shaping, 因为它不改变 r^{step} 、return 或 advantage。

当前 StepPPO-v2 的 WebShop 脚本是:

```
algorithm.use_kl_in_reward=False
```

并继承:

```
actor_rollout_ref.actor.use_kl_loss=True
```

所以当前 v2 中 KL 是 actor loss 正则, 不是 r^{step} 的 reward shaping。当前 v2 的 r^{step} 至少应该包含环境 step reward 和 invalid action penalty; 不需要包含 actor KL loss。

如果后续打开 `algorithm.use_kl_in_reward=True`, 则 StepPPO 需要额外小心: 现有 `apply_kl_penalty` 主要更新 `token_level_rewards`, 但 StepPPO 的 GAE 使用的是 `step_rewards`。因此, 若 KL penalty 被放入 reward 空间, 就必须把每个 step response token 上的 KL penalty 聚合回对应 `step_rewards`:

$$r_{i,t}^{\text{step}} = r_{i,t}^{\text{env}} - \alpha \mathbf{1}\{a_{i,t} \text{ invalid}\} - \beta \sum_{h \in a_{i,t}} D_{\text{KL},i,t,h}.$$

否则会出现 token-level episode advantage 看到了 KL penalty, 而 step GAE return 没看到 KL penalty 的不一致。

为什么不能读取未惩罚 raw episode reward

当前最需要避免的是: A^{step} 使用 shaped step reward, 但 A^{epi} 仍从未惩罚的 raw episode_rewards 读取。这样会导致:

$$A_{i,t}^{\text{final}} = A_{i,t}^{\text{epi,raw}} + w A_{i,t}^{\text{step,shaped}},$$

其中 episode term 可能奖励一个最终得分不错但中间 invalid action 很多的 trajectory, 而 step term 试图惩罚 invalid action。两个信号方向冲突时, final advantage 会变得不可靠。

更一致的做法是定义 shaped episode score:

$$R_i^{\text{epi,shaped}} = \sum_{t=1}^{T_i} r_{i,t}^{\text{step,shaped}},$$

然后 episode advantage 也从 $R_i^{\text{epi,shaped}}$ 计算, 而不是从 raw episode_rewards 计算。

当前 StepPPO 的具体含义

对当前 v2 配置, reward shaping 具体指:

- 必须包含: 环境 step reward $r_{i,t}^{\text{env}}$ 。
- 必须包含: invalid action penalty, 即 $-0.1 \cdot \mathbf{1}\{a_{i,t} \text{ invalid}\}$ 。
- 当前不包含: reward-space KL penalty, 因为 `algorithm.use_kl_in_reward=False`。
- 当前不包含: actor KL loss, 因为它是额外 objective, 不进入 reward/return。
- 如果未来加入 process reward、step bonus、tool cost、length penalty、safety penalty 等, 只要它们用于改变 advantage/return, 就也必须进入 r^{step} 和 shaped episode score。

一句话总结: 这里的 reward shaping 不是泛泛地“奖励工程”, 而是指所有进入 r^{step} 、return、advantage 的 reward-space 修正。当前 v2 最重要的是保证 invalid action penalty 同时进入 step GAE 和 episode advantage; KL 只有在 `use_kl_in_reward=True` 时才需要进入 r^{step} , 否则它只是 actor KL loss。

Turn 4 | User

和 GiGPO 对齐吧, A_{epi} 的做法。然后 A_{step} 和 GiGPO 的 step adv 同样做法, 这样如何

Turn 4 | Codex

可以, 但需要先明确这个实验的算法性质: 如果 A^{epi} 和 A^{step} 都完全按 GiGPO 做, 那么核心 advantage 已经是 GiGPO-style relative advantage; 剩下的差异只来自是否额外保留 value head/value loss, 以及 actor loss path 是否仍与 GiGPO 完全一致。此时它不再是严格意义上的 value-based Step-PPO/Turn-PPO。

对齐后的 advantage 形式是:

$$A_{i,t}^{\text{final}} = A_{i,t}^{\text{epi,GiGPO}} + w A_{i,t}^{\text{step,GiGPO}}.$$

其中 episode term 为:

$$A_{i,t}^{\text{epi,GiGPO}} = \text{Norm}_{\text{uid}} \left(R_i^{\text{epi,shaped}} \right),$$

step term 为:

$$A_{i,t}^{\text{step,GiGPO}} = \text{Norm}_{\text{anchor}(s_{i,t}), \text{uid}} \left(G_{i,t}^{\text{step,shaped}} \right).$$

这里的 $G_{i,t}^{\text{step,shaped}}$ 是 GiGPO 的 step discounted return, 而不是 critic GAE:

$$G_{i,t}^{\text{step,shaped}} = \sum_{k=t}^{T_i} \gamma^{k-t} r_{i,k}^{\text{step,shaped}}.$$

这意味着不再使用 critic TD-error:

$$\delta_{i,t} = r_{i,t} + \gamma V(s_{i,t+1}) - V(s_{i,t}),$$

也不再使用 value-based GAE:

$$A_{i,t}^{\text{GAE}} = \sum_{l \geq 0} (\gamma \lambda)^l \delta_{i,t+l}.$$

因此, 这个版本更准确地应称为 GiGPO-aligned advantage 诊断实验, 而不是 Step-PPO 主算法。

这样做的价值

这个实验很有诊断价值, 因为它可以排除当前 StepPPO-v2 中两个主要不一致点:

1. A^{epi} 的 reward 口径不一致。当前 StepPPO-v2 可能从 raw `episode_rewards` 读取 episode score, 而 GiGPO 从 `shaped token_level_rewards.sum(-1)` 计算 episode advantage。
2. A^{step} 的定义和归一化不一致。当前 StepPPO-v2 用 value-based GAE 后做 batch-level whitening; GiGPO 用 step discounted return, 并在 `anchor_obs + uid` 定义的 step group 内做 relative normalization。

如果这个 GiGPO-aligned 版本恢复到 GiGPO 的表现, 说明当前 v2 的失败主要来自 value-based GAE、value loss 或 reward 口径不一致。如果它仍然不稳定, 则需要继续检查 actor update path、token-level clipping、value auxiliary loss、micro-batch 或其它工程差异。

是否保留 value head

需要拆开两个版本, 否则实验含义会混在一起。

第一个版本是 **GiGPO-aligned-no-value**:

- A^{epi} 完全使用 GiGPO 做法;
- A^{step} 完全使用 GiGPO 做法;

- 关闭 value head 和 value loss;
- 这个版本应当接近 GiGPO baseline, 用于验证对齐是否正确。

第二个版本是 **GiGPO-aligned-value-aux**:

- advantage 仍完全使用 GiGPO;
- 保留 value head, 但 value head 只作为 auxiliary value regression;
- critic 不参与 advantage;
- 建议先设置 detach_value_backbone=True, 避免 auxiliary value loss 干扰 actor backbone;
- value_loss_coef 保持较小, 例如 0.01 或 0.03。

如果保留 value head 且 detach_value_backbone=False, 则算法变成:

$$\mathcal{L} = \mathcal{L}_{\text{GiGPO policy}} + c_v \mathcal{L}_V,$$

且 $\mathcal{L}_V$ 会反传到 shared actor backbone。这样即使 advantage 已经对齐 GiGPO, value loss 仍可能改变 actor 表征并影响 action format 学习。因此不建议把这个混合版本作为第一个诊断实验。

与真正 Step-PPO/Turn-PPO 的关系

如果目标是研究 Step-PPO/Turn-PPO, 那么最终主线仍应使用 value-based GAE:

$$A_{i,t}^{\text{step}} = \sum_{l \geq 0} (\gamma \lambda)^l (r_{i,t+l} + \gamma V(s_{i,t+l+1}) - V(s_{i,t+l})).$$

GiGPO-aligned 版本不能证明 critic-based Step-PPO 有效; 它只能作为 diagnostic control, 用来定位当前失败到底来自 reward/normalization 不一致, 还是来自 value head/value loss 本身。

建议命名

建议新增两个诊断脚本:

- v3_gigpo_aligned_no_value: 完全 GiGPO advantage, 无 value head。
- v3_gigpo_aligned_value_aux: 完全 GiGPO advantage, 加 auxiliary value loss。

其中 v3_gigpo_aligned_no_value 应作为第一优先级。如果它不能复现 GiGPO 附近的结果, 说明实现还没有真正对齐; 如果它能复现, 再加入 value_aux 检查 value head 是否会破坏稳定性。

Turn 5 | User

target 应该是 StepPPO 的那个 GAE return。为什么不这么做

两套方案都先说一下怎么做吧, 先记录到 003 中

Turn 5 | Codex

这里需要区分两套 value-aux target 方案。二者都保持 GiGPO policy advantage 不变:

$$A_{i,t}^{\text{policy}} = A_{i,t}^{\text{epi, GiGPO}} + w A_{i,t}^{\text{step, GiGPO}}.$$

区别只在 value head 回归什么 target。

方案 A: 回归 GiGPO step return

这是当前 `gigpo_v1_value_aux` 里最小改动的实现方式。GiGPO 原本会先计算 step discounted return:

$$G_{i,t}^{\text{GiGPO}} = \sum_{k=t}^{T_i} \gamma^{k-t} r_{i,k}.$$

然后 GiGPO 用它做 step relative advantage:

$$A_{i,t}^{\text{step,GiGPO}} = \text{Norm}_{\text{anchor}(s_{i,t}), \text{uid}} (G_{i,t}^{\text{GiGPO}}).$$

方案 A 直接把这个 GiGPO step return 作为 value target:

$$Y_{i,t}^A = G_{i,t}^{\text{GiGPO}}.$$

实现上, 在 GiGPO advantage 计算完成后补:

```
data.batch["step_returns"] = data.batch["step_rewards"].detach().clone()
```

然后复用 shared value-head actor path 里的 clipped value loss:

$$\mathcal{L}_V^A = \frac{1}{2} \max \left((V_\psi(s_{i,t}) - \text{stopgrad}(Y_{i,t}^A))^2, (\text{clip}(V_\psi(s_{i,t}), V_{\text{old}}(s_{i,t}) - \epsilon_v, V_{\text{old}}(s_{i,t}) + \epsilon_v) - \text{stopgrad}(Y_{i,t}^A))^2 \right).$$

这个方案的优点是实现非常小, 不改变 GiGPO advantage, 也不需要重建 step order 或 GAE。它测试的是: value head 是否能作为 auxiliary task 回归 GiGPO 已经使用的 step signal, 以及这个 auxiliary loss 是否会破坏 GiGPO 稳定性。

但它的限制也很明确:

- 它不是 PPO/StepPPO critic target, 因为没有使用 bootstrap value, 也没有使用 TD-error/GAE。
- 它更像 Monte-Carlo step return regression, critic 不参与 advantage。
- 当前 GiGPO pipeline 是先用 raw immediate reward 计算 discounted return, 再应用 invalid action penalty 到 `step_rewards` 当前 row; 严格来说, 未来 step 的 invalid penalty 没有折扣传播回更早的 return。
- 如果目标是判断 value head 能否成为真正 critic, 方案 A 的证据较弱。

因此, 方案 A 是一个低风险诊断: **GiGPO policy + auxiliary regression to GiGPO step signal**。它可以回答“加 value head/value loss 是否会破坏 GiGPO”, 但不能充分回答“StepPPO critic 是否可行”。

方案 B: 回归 StepPPO GAE return

这是你指出的更合理的 critic-oriented 方案。policy advantage 仍然保持 GiGPO 不变, 但 value head 的 target 改为 StepPPO 那套 GAE return:

$$\delta_{i,t} = r_{i,t}^{\text{step,shaped}} + \gamma V_{\text{old}}(s_{i,t+1}) - V_{\text{old}}(s_{i,t}),$$

$$A_{i,t}^{\text{GAE}} = \sum_{l \geq 0} (\gamma \lambda)^l \delta_{i,t+l},$$

$$Y_{i,t}^B = \hat{R}_{i,t}^{\text{GAE}} = A_{i,t}^{\text{GAE}} + V_{\text{old}}(s_{i,t}).$$

value loss 变成:

$$\mathcal{L}_V^B = \frac{1}{2} \max \left((V_\psi(s_{i,t}) - \text{stopgrad}(\hat{R}_{i,t}^{\text{GAE}}))^2, (\text{clip}(V_\psi(s_{i,t}), V_{\text{old}}(s_{i,t}) - \epsilon_v, V_{\text{old}}(s_{i,t}) + \epsilon_v) - \text{stopgrad}(\hat{R}_{i,t}^{\text{GAE}}))^2 \right).$$

实现上需要做几件事:

1. old-log-probability 阶段仍然让 value head 输出 `state_values`, 这个值在 `torch.no_grad()` 下计算, 是 $V_{\text{old}}(s_{i,t})$ 。
2. 在 batch 被 `adjust_batch` 复制前, 为每个真实 step 记录 `step_idx` 和 duplicate-safe 的 `step_sample_uid`, 避免 padding/复制样本被 GAE 当成额外 environment transition。
3. 为 value target 单独构造 shaped immediate reward $r_{i,t}^{\text{step,shaped}}$ 。不能直接用 GiGPO 的 `step_rewards`, 因为那已经是 discounted return, 而不是 immediate reward。
4. 当前配置下 shaped immediate reward 至少应为

$$r_{i,t}^{\text{step,shaped}} = r_{i,t}^{\text{env}} - 0.1 \cdot \mathbf{1}\{a_{i,t} \text{ invalid}\}.$$

如果未来打开 `algorithm.use_kl_in_reward=True`, 还要把 step response token 上的 KL penalty 聚合进 $r_{i,t}^{\text{step,shaped}}$ 。

5. 用 StepPPO 的 GAE helper 计算 A^{GAE} 和 $\hat{R}^{\text{GAE}}$, 但只把 $\hat{R}^{\text{GAE}}$ 写入 `step_returns` 作为 value loss target; policy loss 的 `advantages` 仍然保留 GiGPO 的 A^{policy} 。

这个方案的优点是:

- value head 的训练目标和 StepPPO/Turn-PPO critic 语义一致;
- 能直接测试 value head 是否能学习 bootstrapped step return;
- 如果这个 auxiliary target 稳定, 而 StepPPO-v2 不稳定, 就说明问题更可能出在“用 GAE advantage 更新 policy”而不只是 value head 本身。

它的限制是:

- 实现比方案 A 更侵入, 需要额外维护 immediate shaped reward、step order 和 duplicate-safe id;
- policy advantage 与 value target 不再来自同一个 estimator: policy 用 GiGPO relative advantage, value 用 GAE return, 所以它仍然不是标准 PPO, 只是更接近 critic feasibility test;
- 如果 `detach_value_backbone=False`, value loss 仍可能通过 shared backbone 影响 GiGPO policy 表征。

为什么一开始没有直接做方案 B

方案 A 是最小诊断补丁: 复用 GiGPO 已存在的 `step_rewards`, 只验证“加 value head/value loss 会不会破坏 GiGPO”。但你指出的是对的: 如果我们真正想测试 StepPPO critic 是否可行, target 应该是方案 B 的 $\hat{R}^{\text{GAE}}$, 而不是 GiGPO step return。

因此后续更合理的实验顺序是:

1. 先跑方案 A, 回答最小问题: auxiliary value loss 是否破坏 GiGPO 稳定性。
2. 再跑方案 B, 回答更关键的问题: shared value head 能否学习 StepPPO/Turn-PPO 风格的 GAE return。
3. 若方案 B 稳定, 再考虑让 A^{GAE} 进入 policy advantage; 否则不要直接把 GAE advantage 加进 policy。

命名上建议:

- `gigpo_v1_value_aux_return_gigpo`: 方案 A。

- `gigpo_v1_value_aux_return_gae`: 方案 B。

一句话结论：方案 A 适合测试 value loss 是否破坏 GiGPO；方案 B 才适合测试 StepPPO critic target 是否可行。若只能选一个与 StepPPO 目标更一致的版本，应优先实现方案 B。

Turn 6 | User

你应该使用的是方案 B 这种实现方式

Turn 6 | Codex

已将 `gigpo_v1_value_aux` 的实现口径修正为方案 B: policy advantage 仍然完全使用 GiGPO, 但 auxiliary value loss 的 target 改为 StepPPO-style GAE return, 而不是 GiGPO step discounted return。

修正后的实现语义为:

$$A_{i,t}^{\text{policy}} = A_{i,t}^{\text{epi,GiGPO}} + w A_{i,t}^{\text{step,GiGPO}},$$

但 value head target 为:

$$\begin{aligned}\delta_{i,t} &= r_{i,t}^{\text{step,shaped}} + \gamma V_{\text{old}}(s_{i,t+1}) - V_{\text{old}}(s_{i,t}), \\ A_{i,t}^{\text{GAE}} &= \sum_{l \geq 0} (\gamma \lambda)^l \delta_{i,t+l}, \\ \hat{R}_{i,t}^{\text{GAE}} &= A_{i,t}^{\text{GAE}} + V_{\text{old}}(s_{i,t}).\end{aligned}$$

然后写入:

$$\text{data.batch["step_returns"]} = \text{stopgrad}(\hat{R}_{i,t}^{\text{GAE}})$$

触发 shared value-head path 中的 clipped value loss。

具体实现点如下:

- 新增 trainer 仍为 `verl/trainer/ppo/gigpo_v1_value_aux_trainer.py`。
- 原始 GiGPO advantage 不变, 仍由原始 `compute_gigpo_outcome_advantage` 生成。
- 在 `adjust_batch` 前补 `step_idx` 和 `step_sample_uid`, 避免 batch padding/copy 产生伪 transition。
- value target 单独从 raw per-step `non_tensor_batch["rewards"]` 构造 shaped immediate reward, 而不是复用 GiGPO 的 discounted `step_rewards`。
- 当前 shaped immediate reward 至少包含 invalid action penalty:

$$r_{i,t}^{\text{step,shaped}} = r_{i,t}^{\text{env}} - \alpha \mathbf{1}\{a_{i,t} \text{ invalid}\}.$$

- 如果未来打开 `algorithm.use_kl_in_reward=True`, 实现会把 `token_level_rewards - token_level_scores` 的 token sum 作为 reward-space KL delta 加回当前 step immediate reward。
- 配置已改为:

$$\text{algorithm.gigpo_v1_value_aux.value_target} = \text{"gae_returns"}$$

因此, 当前 `gigpo_v1_value_aux` 不再是方案 A, 而是:

$$\mathcal{L} = \mathcal{L}_{\text{GiGPO policy}} + c_v \mathcal{L}_V^{\text{GAE-return}}.$$

它仍不是标准 PPO, 因为 policy advantage 仍为 GiGPO; 但它更适合测试 shared value head 是否能学习 StepPPO/Turn-PPO 风格的 critic target。

Turn 7 | User

接下来我有个想法，GiGPO是使用硬文本匹配来计算step的advantage的。有没有一种其他方法？

→ 比如模型的既视感之类的，来对各个step之间进行软匹配，从而实现估计。

→ 这个的好处是不需要硬性匹配了。

请你草拟这个想法为一个文档

Turn 7 | Codex

Codex 将这个想法整理为一个新的 GiGPO 扩展方向，暂命名为 **Soft-GiGPO / Semantic Step Matching**。核心观点是：当前 GiGPO 的 step group 默认依赖 exact anchor observation match，可选 `SequenceMatcher` 也仍然是阈值式 hard clustering；新的设想把“相同 state”的硬文本定义放宽成“相似 state”的连续语义定义。

文档中定义了 soft matching 的基本形式。给定 step 表示 $z_i = f_{\text{match}}(s_i)$ 和相似度 $\text{sim}_{ij} = \cos(z_i, z_j)$ ，在同一 task uid 内构造 leave-one-out softmax 权重：

$$w_{ij} = \frac{\mathbf{1}[u_i = u_j] \mathbf{1}[i \neq j] \exp(\text{sim}_{ij}/\tau)}{\sum_{k: u_k = u_i, k \neq i} \exp(\text{sim}_{ik}/\tau) + \epsilon}.$$

再用加权 baseline

$$b_i^{\text{soft}} = \sum_j w_{ij} R_j$$

得到 soft step advantage：

$$A_i^{\text{soft}} = R_i - b_i^{\text{soft}}.$$

文档建议第一版只做 **Soft-GiGPO-v0**：不改变 policy loss、不接入 StepPPO-v3 value head、不改变 rollout recipe，只替换或增强 GiGPO 的 step relative advantage estimator。候选匹配表示包括 text embedding、actor selected hidden state、frozen actor snapshot，以及后续更高风险的 value-aware similarity。

新增文档位置：

- EXPS/019_soft_gigpo_semantic_step_matching.md
- EXPS/tex/019_soft_gigpo_semantic_step_matching.tex
- EXPS/019_soft_gigpo_semantic_step_matching.pdf